

PROJECT REPORT

Smart Campus Facility & Lab Booking System (CFBS)

Submitted by:

Name: Aniket Thakur

Registration no: 25BAI11190

Academic year: 2026-2027

Subject: CSE2006 - Programming in Java

GitHub Repository:

<https://github.com/thakuraniketsingh16-cloud/CampusFacilityBooking>

1. Introduction

The **Smart Campus Facility & Lab Booking System (CFBS)** is a comprehensive desktop application developed to streamline the administration and reservation of shared physical infrastructure within educational institutions. Built purely using Java SE and Java Swing, the application replaces manual logbooks with a centralized, automated digital dashboard. It facilitates the scheduling of high-tech computing labs, grand auditoriums, and sports arenas while enforcing strict, role-based business rules to ensure fair usage across the campus.

2. Problem Statement

Manual management of shared physical infrastructure in modern educational institutions often leads to administrative inefficiencies. Common challenges include frequent double-bookings, the inability to systematically enforce fair-usage time quotas (e.g., preventing a student from monopolizing a lab all day), and unauthorized access to restricted facilities (e.g., students booking grand auditoriums without faculty endorsement). There is a critical need for an automated, rule-driven system to schedule, validate, and manage these resources seamlessly without the massive overhead of complex enterprise software.

3. Functional Requirements

- **User Authentication & Profiles:** The system must allow users to operate under specific profiles (Student vs. Faculty).
- **Facility Catalog:** The system must display a list of all available facilities, their capacities, types, and specific hardware/features.
- **Booking Engine:** The system must allow users to select a facility, date, and time slot to create a reservation.
- **Collision Detection:** The system must strictly reject overlapping bookings for the same facility.
- **Quota Enforcement:** The system must reject student bookings exceeding 2 hours and faculty bookings exceeding 6 hours per session.
- **Auditorium Access Control:** The system must prevent students from directly booking grand auditoriums (capacity > 150) without faculty endorsement.
- **Cancellation Module:** Users must be able to cancel their own active bookings.
- **Analytics & Reporting:** The system must generate real-time utilization reports grouped by facility and department.

4. Non-functional Requirements

- **Usability:** The interface must feature a modern, intuitive Dark Mode GUI for reduced eye strain and easy navigation.
- **Performance:** File read/write operations must be optimized to ensure instant UI responsiveness during booking validations.
- **Reliability:** Data must be persistently saved to local storage immediately upon any modification to prevent data loss.
- **Portability:** The application must run on any OS (Windows, macOS, Linux) with a standard Java Runtime Environment (JRE 8+) without requiring external database installations.

5. System Architecture

The application employs a monolithic desktop architecture utilizing a lightweight Model-View-Controller (MVC) pattern contained within a unified codebase:

- **Model (DataStore, Facility, User, Booking):** Handles data encapsulation, flat-file I/O operations (campus_data.txt), and core business logic (collision detection).
 - **View (CampusBookingGUI):** Built with Java Swing, overriding the UIManager to render a custom CrossPlatform Dark Theme.
 - **Controller (Action Listeners):** Bridges user inputs from the GUI to the DataStore methods, returning formatted results or custom exceptions (CampusBookingException).
-

6. Design Diagrams

6.1 Use Case Diagram

```
flowchart LR
    Student([Student])
    Faculty([Faculty Member])

    subgraph CFBS_System [CFBS System]
        B1((B1 View Facility Catalog))
        B2((B2 Book Computing Lab / Court))
        B3((B3 Book Grand Auditorium))
        B4((B4 Cancel Own Booking))
        B5((B5 View Analytics))
    end

    Student --> B1
    Student --> B2
    Student --> B4
    Student --> B5

    Faculty --> B1
    Faculty --> B2
    Faculty --> B3
    Faculty --> B4
    Faculty --> B5
```

6.2 Workflow Diagram

```
flowchart TD
    A[Launch Application] --> B[Auto-load DataStore]
    B --> C[Select Active Profile]
    C --> D{Choose Action}
    D -->|View Catalog| E[Display Facilities]
    D -->|New Booking| F[Enter Booking Details]
    F --> G{Validation Engine}
    G -->|Overlap/Quota Error| H[Throw Exception & Reject]
    G -->|Valid| I[Generate Booking ID]
    I --> J[Save to flat-file]
    J --> K[Show Success Confirmation]
```

6.3 Sequence Diagram

```
sequenceDiagram
    actor User
    participant GUI as CampusBookingGUI
    participant Store as DataStore
    participant File as campus_data.txt

    User->>GUI: Clicks "Reserve Slot"
    GUI->>Store: createBooking(User, Facility, Date, Times)
    activate Store
    Store->>Store: Validate Quota & Permissions
    Store->>Store: Check Overlaps
    alt Invalid Request
        Store-->>GUI: throws CampusBookingException
        GUI-->>User: Show Error Dialog
    else Valid Request
        Store->>Store: Instantiate Booking Object
        Store->>File: save() (Write to Disk)
        Store-->>GUI: Returns Booking Object
        GUI-->>User: Show Success Message
    end
    deactivate Store
```

6.4 Class Diagram

```
classDiagram
    class User {
        <<abstract>>
        -String id
        -String name
        +getMaxBookingHours()* int
        +canBookAuditoriumDirectly()* boolean
    }
    class Student {
        -String regNo
    }
    class FacultyMember {
        -String empId
    }

    class Facility {
        <<abstract>>
        -String id
        -String name
        -int capacity
        +getSpecificDetails()* String
    }
    class Lab {
        -int workstations
        -boolean hasGPU
    }
```

```

}
class Hall {
    -boolean hasProjector
}
class SportsCourt {
    -boolean indoor
}

class Booking {
    -String bookingId
    -LocalDate date
    -LocalTime startTime
    -LocalTime endTime
    +overlaps(date, start, end) boolean
}

User <|-- Student
User <|-- FacultyMember
Facility <|-- Lab
Facility <|-- Hall
Facility <|-- SportsCourt

DataStore "1" *-- "many" User
DataStore "1" *-- "many" Facility
DataStore "1" *-- "many" Booking

```

6.5 ER Diagram (Flat-File Representation)

```

erDiagram
    USER ||--o{ BOOKING : places
    FACILITY ||--o{ BOOKING : hosts

    USER {
        string ID PK
        string Role
        string Name
        string Department
    }
    FACILITY {
        string ID PK
        string Type
        int Capacity
        string Location
    }
    BOOKING {
        string BookingID PK
        string UserID FK
        string FacilityID FK
        date Date
        time StartTime
        time EndTime
        string Status
    }

```

7. Design Decisions & Rationale

1. **Single-File Architecture:** To satisfy strict academic submission constraints (ensuring it can be compiled out-of-the-box via a single `javac Main.java` command).
2. **Flat-File Persistence (`campus_data.txt`):** Chosen over JDBC/MySQL to guarantee 100% portability. Anyone testing the project doesn't need to configure a database server; the system automatically seeds its own demo data if the file is missing.
3. **Java Swing with Custom UIManager:** Selected over JavaFX because Swing is deeply integrated into standard JDKs. A manual Dark Mode was engineered by overriding `UIManager` properties globally to achieve a modern aesthetic without third-party libraries (like FlatLaf).
4. **Java 8 Stream API:** Used extensively in the Analytics module to aggregate, group, and sort large collections of bookings concisely and efficiently.

8. Implementation Details

- **Core Concepts Used:** The project rigorously applies Object-Oriented Programming (OOP). `Facility` and `User` are abstract classes enforcing polymorphism (e.g., dynamically resolving `getMaxBookingHours()` based on whether the object is a `Student` or `FacultyMember`).
- **Exception Handling:** A custom hierarchy of checked exceptions (`CampusBookingException`, `SlotUnavailableException`, `InvalidInputException`) ensures predictable error recovery.
- **Date & Time API:** Utilizes the modern `java.time.LocalDate` and `LocalTime` classes for robust interval math and overlap detection.

9. Screenshots / Results

(Note: Replace these placeholders with actual screenshots from the application)

1. Dashboard & Analytics

[Insert Screenshot of the Analytics Tab here showing the Dark Mode UI and utilization stats]

2. Facility Booking & Validation

[Insert Screenshot of the New Booking Tab demonstrating a successful reservation or a collision error]

10. Testing Approach

- **Automated Self-Diagnostics:** A built-in class (`TestHarness`) can be triggered via `java Main --test`. It runs headless boundary tests simulating quota overflows, time overlaps, and auditorium permission denials to ensure core logic integrity without human intervention.
- **Manual GUI Testing:** Exhaustive black-box testing was conducted by creating conflicting bookings through the UI, verifying that error messages format correctly, and confirming that cancellations immediately free up the corresponding time slots.

11. Challenges Faced

- **Dark Mode Implementation:** Forcing legacy Java Swing components (like `JTable` headers and `JScrollPane` borders) to conform to a dark theme natively required deep manipulation of obscure `UIManager` properties.
- **Collision Logic:** Designing a mathematically sound overlap detection algorithm that handles edge cases (e.g., back-to-back bookings where `EndTime A` exactly equals `StartTime B`) required careful use of `.isBefore()` and `.isAfter()` constraints.
- **Single-File Clutter:** Managing Model classes, Data I/O, custom Exceptions, and complex GUI layouts within a single `Main.java` file demanded extremely strict structural organization and commenting.

12. Learnings & Key Takeaways

- Mastered the implementation of Java 8 Stream API for complex data aggregations and filtering.
- Gained profound understanding of the `java.time` package and temporal logic.
- Learned how to effectively decouple business logic (Validation Engine) from the presentation layer (Swing GUI) even within tightly constrained single-file architectures.
- Developed a strong grasp of file I/O operations and robust data-parsing techniques.

13. Future Enhancements

- **Database Integration:** Upgrading the storage backend from flat-file I/O to a robust relational database (MySQL/PostgreSQL) using JDBC.
- **Email Notifications:** Integrating the JavaMail API to send automated booking confirmations and cancellation receipts to users.
- **Calendar View UI:** Replacing the text-based schedule viewer with an interactive, graphical drag-and-drop weekly calendar matrix.

14. References

1. Oracle Java SE Documentation: <https://docs.oracle.com/javase/8/docs/api/>
2. Creating a GUI With Swing (The Java™ Tutorials): <https://docs.oracle.com/javase/tutorial/uiswing/>
3. Project Source Code Repository: <https://github.com/thakuraniketsingh16-cloud/CampusFacilityBooking>